# History-Based Data Import and Version Restore

**Robroy Rep Locator — Functional Plan and Implementation Blueprint**

| Document field | Value |
|---|---|
| Version | 1.0 |
| Prepared | 1 October 2026 |
| Status | Proposed implementation |
| Audience | Client stakeholders, developers, and operations |
| Application | replocator.robroy.com |

This document defines how the client will upload representative data, review changes, publish a version, and restore any previously saved successful version. It includes the functional workflow, technical design, delivery sequence, and acceptance criteria. The described functionality is proposed; this document does not indicate that it has been implemented or deployed.

## 1. Objective and operating rules

Every successful import must produce a complete, immutable version of the representative dataset. The client must be able to inspect any successful version and restore its data without losing later history.

The following rules define completion:

1. Save the existing live dataset as the initial baseline, including an empty baseline when appropriate.
2. Validate and preview each import before publishing it.
3. Publish live data and its saved version as one atomic operation.
4. Preserve the current live dataset when validation or publication fails.
5. Restore a saved version by creating a new successful version.
6. Keep all successful versions available for restoration; provide no routine deletion or expiration of those versions.
7. Record who performed each operation, what changed, and when it completed.
8. Protect all administrative operations with authentication and authorization.

**Example:** V1 is the baseline, V2 and V3 are imports, and the client restores V1. The system creates V4 containing V1's business data and marks V4 current. V1, V2, and V3 remain available. Restoring V3 afterward creates V5.

# 2. Scope and assumptions

## Included in the first release

- Representative contact and address data in `Rep`.
- Brand, state, county, country, and territory assignments in `RepCounty`.
- CSV uploads, validation, change previews, publication, history, comparison, export, and restoration.
- Full snapshots covering every brand after each successful publication.
- Imports that replace selected brands, replace all brands, or add new records.
- Complete dataset restoration across all brands, with affected brands shown before confirmation.
- Retention of the original source file and the normalized saved dataset.

## Boundaries

ZIP reference data in `Zip` is outside the first release's version boundary. Restoring representatives therefore does not reproduce historical ZIP lookup behavior if that reference table has changed. Versioning ZIP data can be added if exact historical search behavior is required.

Map coordinates are derived data. Save the coordinates available at publication for traceability, but exclude them from business-data comparisons and checksums. A restore reproduces saved contacts and territory assignments; subsequent geocoding may refresh coordinates without changing that business version.

Brand-specific restoration is a possible later feature. The first release restores the complete saved dataset to avoid ambiguous combinations of several historical states.

Versions predating the first baseline are available only when recoverable from old snapshots, backups, or source files. Original file retention alone is insufficient for exact restoration because parsing and normalization rules can change.

**Planning assumption:** Full snapshots fit the current dataset size. Measure live row counts, snapshot growth, transaction duration, and restore time before finalizing capacity and timeouts.

# 3. Existing repository and prerequisites

The application uses Express, MySQL, and a static admin page. Existing work provides a foundation, but it does not yet implement this design end to end.

| Component | Observed state and required change |
| --- | --- |
| `app/ImportHistory.js` | Snapshot and comparison methods exist. Add transactional publication, current-version tracking, restore lineage, and immutability. |

| Component | Observed state and required change |
| --- | --- |
| `routes/Import.js` | Only legacy import GET routes are implemented. Add the protected administrative API and remove destructive GET behavior. |
| `app/RepCountyComboImporter.js` | Truncates live tables before loading; its temporary schema omits `rep_country` used by the load and insert. Replace this workflow with staging and atomic publication. |
| `public/admin/index.html` | Calls endpoints that are currently missing. Connect the workflow and add preview, current-version, export, and restore controls. |
| `database.sql` and migrations | Reconcile country fields, rep identity, snapshot fields, and migration order against the actual deployed schema. |
| `app/Geocoder.js` | Prevent delayed results from modifying reps whose addresses or dataset version changed. |

Several of these newer files are currently untracked in the working tree. Include the intended implementation files in the delivery branch before release.

The base schema identifies `Rep` by `rep_code` alone, while the business model requires a representative per brand. Inspect existing data and adopt a consistent `(brand, rep_code)` identity. Territory identity must also include country to avoid collisions between similarly named regions. Align live constraints, staging validation, snapshot constraints, joins, and diff keys with these identities.

Resolve key lengths, character encoding, and existing duplicates before applying constraints. The implementation must support the actual database version and existing production data; do not apply a generic schema replacement to a populated database.

# 4. Client workflow

## Import and publish

1. The administrator uploads a CSV and selects an import mode.
2. The server saves an immutable upload, parses it, validates every row, and builds the complete proposed dataset.
3. The preview shows the source filename, selected brands, current version, resulting counts, warnings, and additions, updates, and removals.
4. Errors identify the row, field, and reason. The administrator corrects and uploads a new file when validation fails.
5. The administrator confirms publication of the previewed candidate.
6. The server verifies that the current version still matches the preview, publishes atomically, and returns the new version number.
7. The admin page refreshes its current-version indicator and history. Public searches use the published data.

| Mode | Meaning |
|---|---|
| Replace selected brands | Replace reps and territories only for explicitly selected brands; retain other brands. Reject rows for unselected brands. |
| Replace all brands | Treat the uploaded dataset as the entire replacement. Show a prominent summary of every affected brand and removal. |
| Add new records | Add new reps or assignments. Existing identical reps may be referenced; conflicting details for an existing identity fail validation. Identical existing assignments are treated as no-ops and counted. |

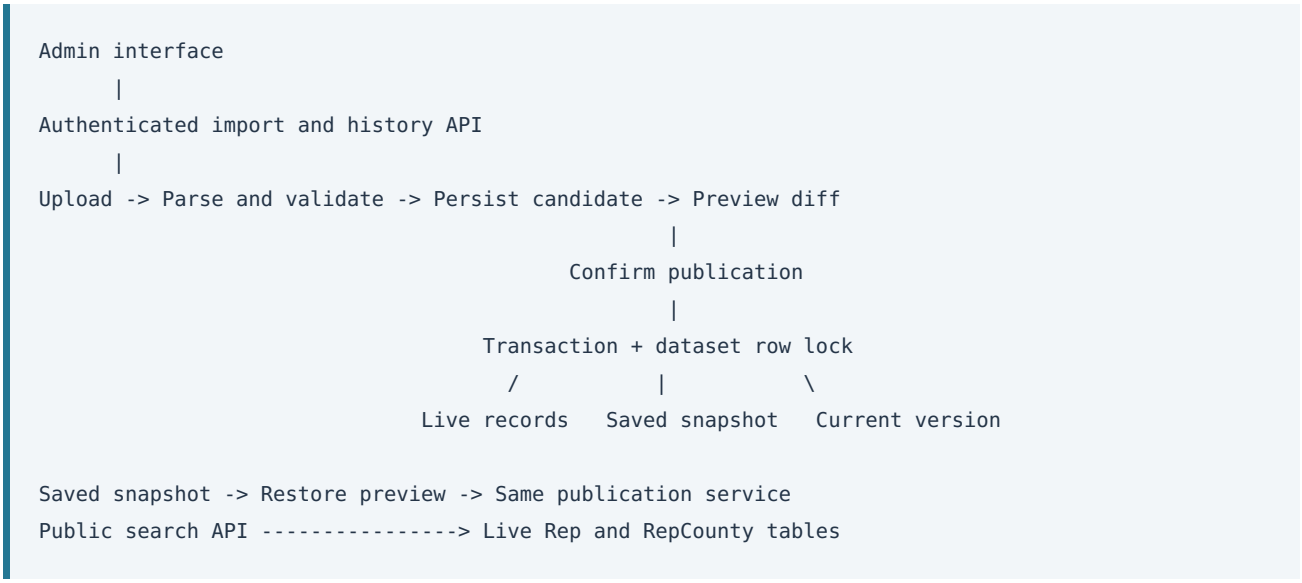
An empty replacement or removal of every rep for a selected brand requires an explicit, separately confirmed clear-data action. An accidentally empty CSV must not erase data.

### Restore

1. The administrator opens a successful history version and chooses **Preview restore**.
2. The system compares that snapshot with the currently published version.
3. The screen identifies the source version, current version, affected brands, record changes, and the fact that restoration covers the entire dataset.
4. The administrator confirms **Restore this version**.
5. The server verifies preview freshness and snapshot integrity, then publishes the saved data as a new version.
6. History displays the new entry as **Restored from Vx** and marks it current.

Failed attempts, unfinished previews, and snapshots with integrity or compatibility errors cannot be restored. Restoring the current version can be disabled as an unnecessary operation. A successful import with no business-data differences can still create a version, labeled **No data changes**, to preserve the import audit trail.

## 5. Architecture and data ownership




Keep the public search API reading the existing live tables. Centralize every change to versioned business data in one publication service. Block legacy import routes and direct administrative edits from bypassing that service.

Candidate data must be durable across application restarts and independent of an HTTP connection. Use persistent staging tables keyed by a preview ID. Avoid relying on a connection-scoped temporary table across pooled queries.

## 6. Proposed data model

This is a logical schema specification. Final SQL migrations must be prepared after inspecting the deployed database and resolving the prerequisites in section 3.

| Entity | Required information |
|---|---|
| `ImportUpload` | ID, original filename, private storage key, SHA-256 file hash, byte size, authenticated actor, upload time. |
| `ImportPreview` | ID, operation and mode, upload or restore-source ID, base version, selected brands, parser/schema version, candidate hash, validation status, counts, creator, expiry. |
| `RepCandidate` / `RepCountyCandidate` | Complete proposed business dataset keyed by preview ID, with identities matching live data. |
| `ImportRun` | Attempt ID, successful version number, operation, mode, status, upload/preview IDs, previous version, restored-from version, actor, timestamps, counts, business-data hash, schema version, error code. |
| `RepSnapshot` / `RepCountySnapshot` | Full saved rows keyed by successful run ID, with uniqueness and lookup indexes. |
| `DatasetState` | One stable dataset row holding current successful run ID, current version number, and update time. |

Separate the operation attempt ID from the visible version number. Failed attempts have no successful version number. Assign the next version number while holding the dataset lock, so concurrent attempts cannot publish the same number.

Use foreign keys where supported and appropriate. Retain references from runs to source uploads and restored-from versions. Index run dates/status, snapshot run IDs and record identities, and candidate preview IDs. Paginate history and detail rows.

Compute snapshot integrity hashes from canonically sorted, normalized business rows with explicit field names and null handling. Avoid hashes based on arbitrary SQL row order or ambiguous string concatenation. Store the normalization/schema version used to calculate each hash.

Published snapshots are immutable through application APIs. Future schema upgrades must provide a tested compatibility adapter or backfill process for every retained snapshot format.

## 7. CSV ingestion and validation

Use a CSV parser that supports quoted commas, embedded newlines, UTF-8 BOMs, and both common line endings. Map supported header names explicitly rather than assuming fixed column positions. Support the existing 17- and 19-column layouts only after their exact mappings are documented against real sample files.

Validation covers:

- Required headers, supported encoding, upload size, and row limits.
- Required brand and rep code; allowlisted brand codes.
- Consistent rep details when many territory rows refer to the same rep.
- Duplicate identities and assignments using the database's comparison rules.
- Country, state, county, and territory field relationships.
- Every territory's reference to a rep in the same brand.
- Expected field lengths, safe null handling, and preservation of ZIP leading zeros.
- Blank territory fields that intentionally represent broader coverage; these must follow documented business rules.

Return blocking errors separately from warnings. Optional missing contact fields may be warnings; contradictory rep identities or invalid references are errors. Store enough validation detail to explain an unsuccessful upload without exposing internal SQL errors.

Retain each upload under a generated storage name outside `public/`. A client-supplied filename must never determine a filesystem path or SQL identifier. Remove the public `?table=` import path. Authorized offline migrations may use a separate controlled tool.

## 8. Atomic publication, concurrency, and recovery

Create an operation attempt and obtain one dedicated database connection. All writes in the publication transaction use that connection. Parsing, expensive validation, and candidate generation occur before the transaction.

```
Authenticate actor and authorize action
Resolve idempotency key to an existing result or create an attempt
Acquire one database connection
BEGIN
  Lock DatasetState row FOR UPDATE
  Verify preview.base_version equals current_version
  Lock preview and verify status, expiry, ownership, and candidate hash
  Allocate the next successful version number
  Copy complete candidate business data into immutable snapshot tables
  Replace live territory and rep rows from the validated candidate
  Check live counts, relationships, and business-data checksum
  Mark attempt successful and link source/previous versions
  Update DatasetState to the new successful version
  Mark preview consumed
COMMIT
Return the committed version and change summary
```

Use transactional `DELETE` and `INSERT` statements in foreign-key-safe order: remove dependent territories first, then reps; insert reps first, then territories. Keep `TRUNCATE`, schema changes, and CSV file loading outside the live-data publication transaction.

If any statement fails, roll back and record a sanitized failure on the attempt after rollback. Readers must never observe a partially replaced dataset. For searches involving multiple SQL statements, capture one consistent read transaction or a consistent version context if required to avoid mixing metadata from two versions.

Only one publication or restore can hold the dataset lock at a time. A preview based on an older current version receives `409 Conflict` and must be regenerated. Apply a bounded lock timeout so a second user sees a clear busy response.

Require an idempotency key for publish and restore requests. If the connection drops after a successful commit, a retry returns the original result instead of creating another version. On startup, reconcile unfinished attempts with committed state and consumed previews before marking abandoned attempts failed. Do not infer failure solely from a lost client connection.

Coordinate geocoding through version/address guards. An update must match the expected dataset generation and address fingerprint, and coordinate with the publication lock for its short write transaction. Keep network calls outside that transaction. Saved snapshots must never be updated by background geocoding.

## 9. API contract

All endpoints below are relative to `/api/import` and require administrative authentication. Response names are proposed and should be used consistently by the UI and service.

| Method and path | Purpose |
| --- | --- |
| `POST /uploads` | Store a CSV; return upload ID and file metadata. |

| Method and path | Purpose |
| --- | --- |
| `POST /previews` | Validate an upload and build a candidate for the selected mode/brands. |
| `GET /previews/:id` | Return validation state, affected brands, counts, and paginated changes. |
| `POST /previews/:id/publish` | Publish an import using the expected base version and an idempotency key. |
| `GET /state` | Return the current version and active operation information. |
| `GET /runs` | Paginated history with status, date, operation, and brand filters. |
| `GET /runs/:id` | Return attempt/version metadata and paginated snapshot records. |
| `GET /runs/:id/diff?against=:id` | Compare two successful versions; return exact totals and paginated changes. |
| `POST /runs/:id/restore-preview` | Prepare a complete restore candidate against the current version. |
| `POST /runs/:id/restore` | Publish a confirmed restore preview with freshness and idempotency checks. |
| `GET /runs/:id/export` | Download a full dataset export and version manifest. |
| `GET /uploads/:id/download` | Download the retained original CSV when authorized. |

Use `401/403` for access failures, `404` for unavailable resources, `409` for stale previews or conflicting operations, and `422` for invalid input data. Return predictable error codes and readable messages. Use `202` and status polling for work that exceeds the chosen request-time budget.

Do not expose a delete endpoint for successful versions. Expired, unpublished previews can be cleaned up under a documented policy. Uploads referenced by successful runs must remain retained.

Administrative authentication should identify an individual operator when the deployment supports it. If the first release uses a shared admin token, record that shared credential identity honestly; a typed name is not verified attribution. Keep authentication on the server, use HTTPS, avoid tokens in query strings, and fail closed when required credentials are missing. Cookie-based sessions require CSRF protection.

## 10. Admin interface and exports

Show the current version prominently with publication time and source. The import page contains file selection, mode, brand scope, validation results, preview, and publication progress.

History displays version number or failed attempt ID, operation, filename, actor, date, counts,

status, and restore origin. Each successful version provides **View**, **Compare**, **Download**, and **Preview restore** actions. The current version has a clear badge. Disable repeated submission while an operation is pending and retain the idempotency key through retries.

The restore confirmation should read, for example: “Restore V12 across all brands. This will create a new version based on V12. Changes from current V18: 8 reps added, 5 removed, and 14 updated.” Show territory changes and affected brands alongside it.

Compare all business fields, including country and notes. Treat a changed record identity as a removal plus addition. Provide exact totals even when detail rows are paginated. Avoid loading every historical snapshot into browser memory.

Export each saved dataset as a manifest plus separate reps and territories files, or an equivalent lossless JSON package. Include version metadata, schema version, counts, and checksums. Offer the original CSV separately; it may represent only one brand or an additive change. Protect spreadsheet-oriented CSV downloads from formula execution while preserving an unmodified lossless export for exact recovery.

## 11. Implementation work packages

| Phase | Main work and repository impact | Exit condition |
|---|---|---|
| 1. Schema and baseline | Inspect deployed schema; resolve identities; add ordered migrations for version, upload, candidate, and state tables; capture baseline. | Baseline reproduces the pre-change business dataset and has a valid checksum. |
| 2. Ingestion and preview | Refactor `RepCountyComboImporter.js`; introduce upload, parser, validation, and preview services. | Supported CSV samples produce accurate complete candidates without modifying live data. |
| 3. Publication service | Add a shared import/restore publisher; refactor `ImportHistory.js` to accept a transaction connection; add state locking and idempotency. | Valid imports commit atomically; failed or concurrent attempts preserve consistency. |
| 4. History and restore | Implement comparison, snapshot verification, restore preview/publication, and export. | Any supported successful version can be restored as a new version. |
| 5. Protected API and UI | Replace legacy routes; connect `public/admin/index.html`; implement authentication, progress, and client error handling. | Client completes upload, preview, publish, compare, and restore without developer access. |
| 6. Release and operations | Run acceptance checks, rehearse backup recovery, document configuration, and deploy with monitoring. | Restore rehearsal passes and operational ownership is documented. |

Suggested new modules are `app/ImportUpload.js`, `app/ImportValidator.js`, `app/ImportPreview.js`, and `app/DatasetPublisher.js`. Their names may change; preserve a single shared publication path for imports and restores.

Update `utils/mysql.js` as needed to expose dedicated connection handling. Update search joins for the agreed brand/rep identity. Update `app/Geocoder.js` and its CLI entry point with publication-aware writes. Maintain installation instructions, `example.env`, and an operator runbook alongside ordered migrations.

An effort estimate should follow measurement of representative CSV sizes, inspection of production schema drift, and selection of the authentication approach. These dependencies can materially change implementation effort.

## 12. Acceptance and verification matrix

| Scenario | Expected result |
|---|---|
| First activation with existing data | One successful baseline matches the original business dataset. |
| First activation with empty data | Empty baseline is saved and restorable. |
| Valid selected-brand import | Selected brands change; all other brands are preserved in live data and the full snapshot. |
| Invalid CSV or inconsistent repeated rep | Actionable validation errors; current version and live records remain unchanged. |
| Add records with conflicting existing identity | Preview fails with a precise conflict explanation. |
| Failure during rep, territory, or snapshot write | Entire publication rolls back; failed attempt is recorded. |
| Two administrators publish concurrently | Operations serialize; stale preview cannot overwrite the winner. |
| Restore an older successful version | New version's business checksum equals the source; later versions remain intact. |
| Restore after an earlier restore | Another new version is created with correct lineage. |
| Restore corrupt or unsupported snapshot | Restore is rejected before live-data changes. |
| Application crash or lost response | Committed state is recoverable; retry does not publish twice. |
| Delayed geocoding result | Old address/version result cannot modify the replacement rep. |
| Search during publication | Each search reads a consistent published dataset with no partial replacement. |
| Unauthorized request or legacy GET import | No import, restore, file access, or mutation is performed. |

| Scenario | Expected result |
| --- | --- |
| Snapshot format migration | Every retained supported version remains verifiable and restorable. |
| Export and disaster recovery | Complete snapshot can be recovered from backup and produces the expected checksum. |

Use service-level checks for parsing, identity, comparison, and checksums; database integration checks for transactions, locking, retries, and restores; and browser checks for the complete client workflow. Run these against an isolated database with representative data. Do not use production records as a destructive test target.

## 13. Rollout, retention, and operations

1. Inventory the actual schema, application runtime, data volume, and existing historical artifacts.
2. Back up the database and retained files, then rehearse recovery in an isolated environment.
3. Deploy migrations and services with publication disabled; retire legacy mutation paths.
4. Capture and verify the baseline under the same serialization rules used for publication.
5. Exercise import, restore, and retry scenarios in staging and review the UI with the client.
6. Enable managed publication and monitor the first import and restore.

Keep every successful version and every referenced source upload. Clean up only expired candidates, abandoned uploads without retained references, and temporary export artifacts. Monitor snapshot storage growth, failed operations, lock waits, publication duration, and backup freshness.

Back up history tables, live tables, state metadata, and uploaded files as a consistent recovery set. Store backups independently of the application host and periodically rehearse a complete restore. Historical versions inside the same database do not protect against loss of that database.

Agree on backup frequency and recovery time with the client before release. Define operational alerts and ownership. Record restore duration with representative data and use those measurements for user-facing expectations.

Application rollback must preserve the historical schema and data. If a release issue occurs, disable publication and restore service availability; do not automatically drop snapshot tables or reactivate the unsafe legacy importer. Data restoration should continue to use the authenticated version workflow when available.

## 14. Decisions to confirm during implementation

The proposed defaults make the first implementation concrete. Confirm these business and operational choices before release:

- Full-dataset restore across all brands, as specified here, or a separately scoped brand-restore feature.
- Whether ZIP reference versions are required for exact historical search behavior.
- Selected-brand replacement as the normal import mode; all-brand replacement remains an explicit choice.
- Authentication through individual operator identities or an initial shared administrative credential.
- Real CSV header mappings, supported country conventions, and intentional blank-territory semantics.
- Upload limits, preview expiry, storage capacity, backup frequency, and measured recovery objectives.

Successful delivery gives the client a complete history of published data, a reviewable import process, and a repeatable way to restore any retained successful version while preserving the audit trail.